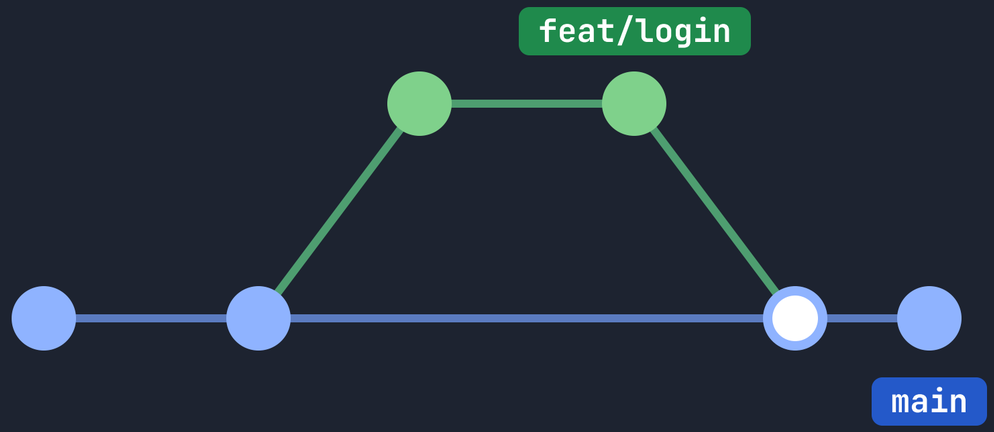
KANT AX/AI 엔지니어 트랙 · 보충 학습 자료

Git 브랜치, 눈으로 보기

따로 작업하고, 합치고,
충돌을 푸는 법



오늘의 순서

1 브랜치란?
점, 이름표, HEAD
3–6쪽

2 만들고 옮기기
branch · checkout
7–13쪽

3 합치기
merge · 이름 규칙
14–18쪽

4 갈라진 길
merge 커밋
19–25쪽

5 충돌 풀기
표시 읽고 고치기
26–32쪽

6 GitHub에서 함께
push · Pull Request
33–38쪽

7 충돌 시나리오
다섯 가지 상황
39–50쪽

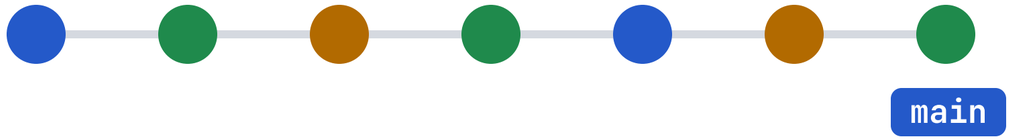
8 막힐 때
자주 나는 에러
51–53쪽

마지막 54쪽은 한 장 요약이에요.

브랜치란?

왜 브랜치를 쓸까요?

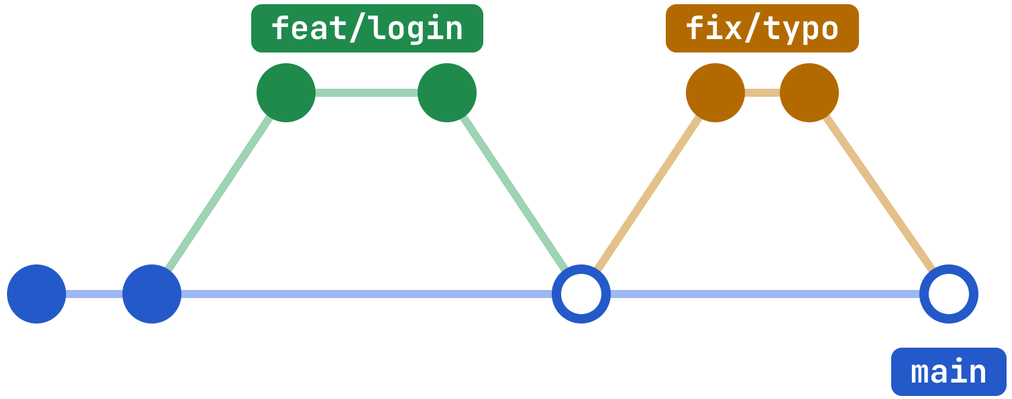
⚠ 모두 main에 바로 커밋



반쯤 만든 기능이 main에 섞여요

누구 작업이 문제인지 찾기 어려워요

✓ 각자 브랜치에서 작업



다 만든 다음에 main으로 합쳐요

main은 늘 잘 돌아가는 상태로 남아요

브랜치 = 본 줄기(main)는 그대로 두고, 옆으로 갈래를 내서 작업하는 방법

점 하나가 커밋 하나예요

```
$ git log --oneline
feff88d (HEAD → main) docs: README 추가
1180617 feat: 인사말 추가
560eba8 feat: 인사 파일 추가
```



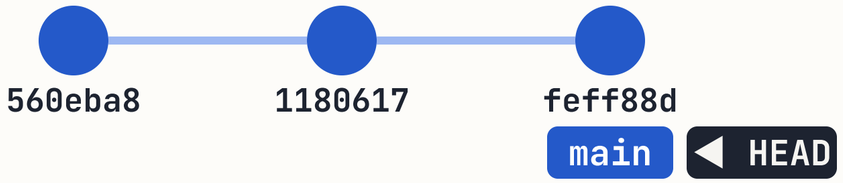
왼쪽이 먼저 한 커밋, 오른쪽이 최근 커밋이에요.

main은 마지막 커밋에 붙은 이름표예요



새 커밋을 하면 이름표가 새 커밋으로 옮겨 가요.

HEAD는 지금 내가 있는 곳이에요

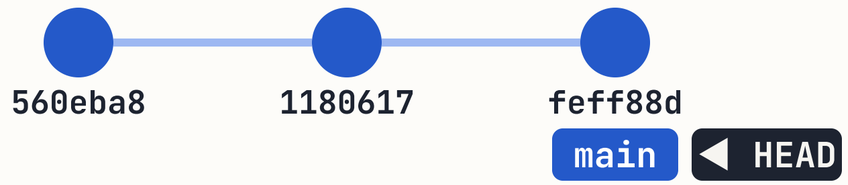


커밋하면 HEAD가 붙은 이름표가 앞으로 가요.

만들고 옮기기

브랜치 목록 보기

```
$ git branch
* main
```



* 표시가 지금 있는 브랜치예요. 그래프의 HEAD와 같은 뜻이에요.

만들고 옮기기

브랜치 만들기: 이름표가 하나 더 생겨요

```
$ git branch feat/greeting
```

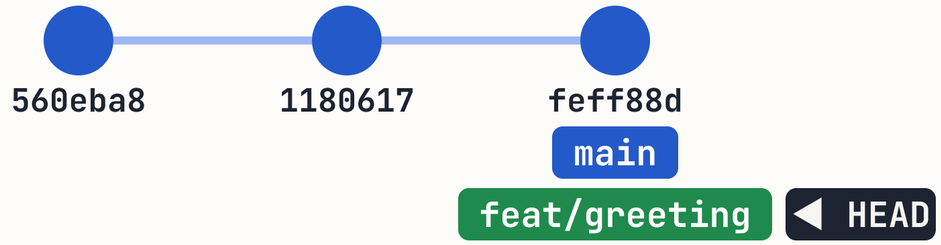


커밋은 복사되지 않아요. HEAD는 아직 main에 있어요.

만들고 옮기기

브랜치로 옮겨 가기: HEAD가 움직여요

```
$ git checkout feat/greeting
Switched to branch 'feat/greeting'
```

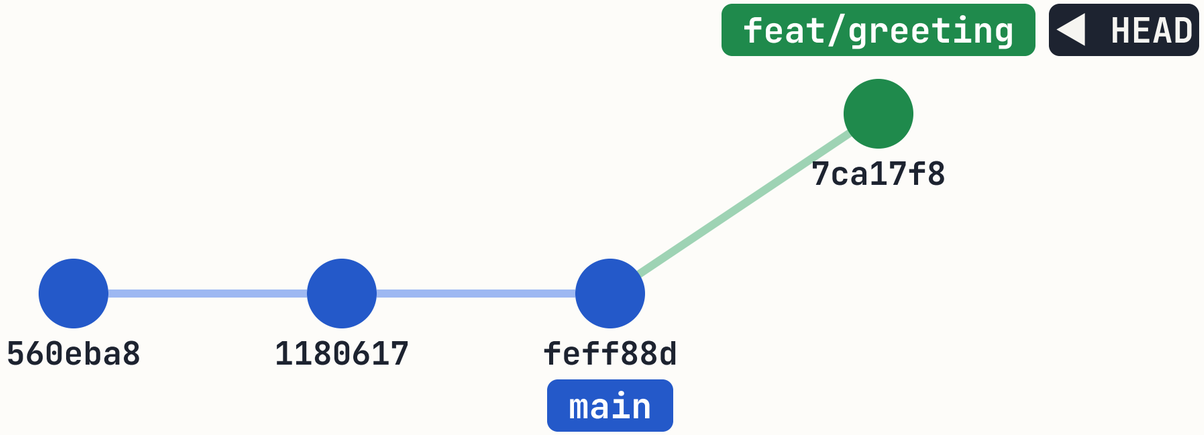


이제부터 하는 커밋은 feat/greeting에 쌓여요.

만들고 옮기기

커밋하면 feat/greeting만 앞으로 가요

```
$ git add hello.md
$ git commit -m "feat: 반가움 인사 추가"
[feat/greeting 7ca17f8] feat: 반가움 인사 추가
```

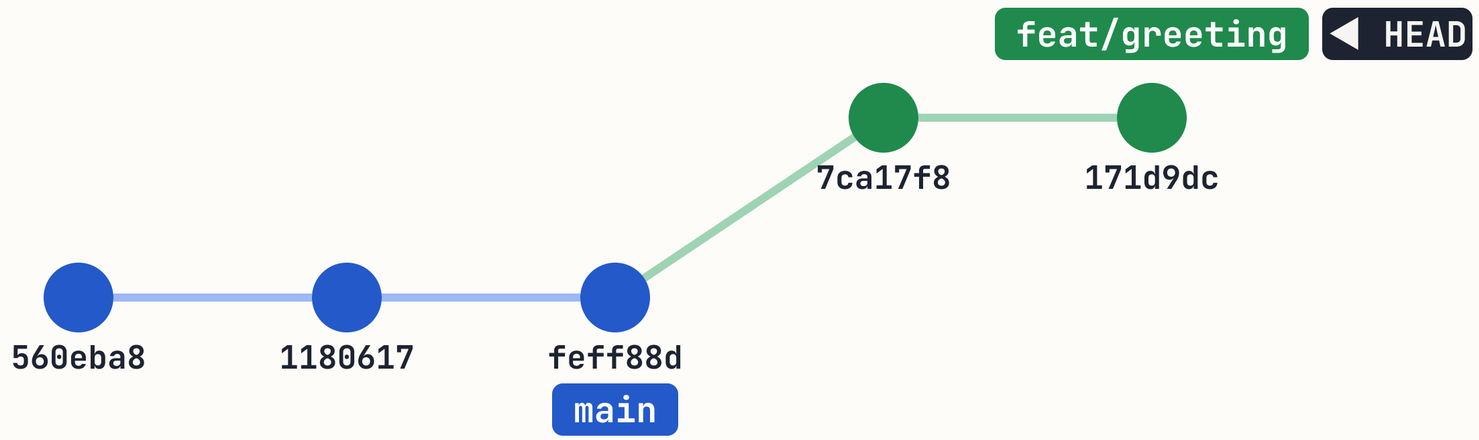


main은 제자리예요.

만들고 옮기기

한 번 더 커밋하면 한 칸 더

```
$ git add bye.md
$ git commit -m "feat: 끝인사 파일 추가"
[feat/greeting 171d9dc] feat: 끝인사 파일 추가
```

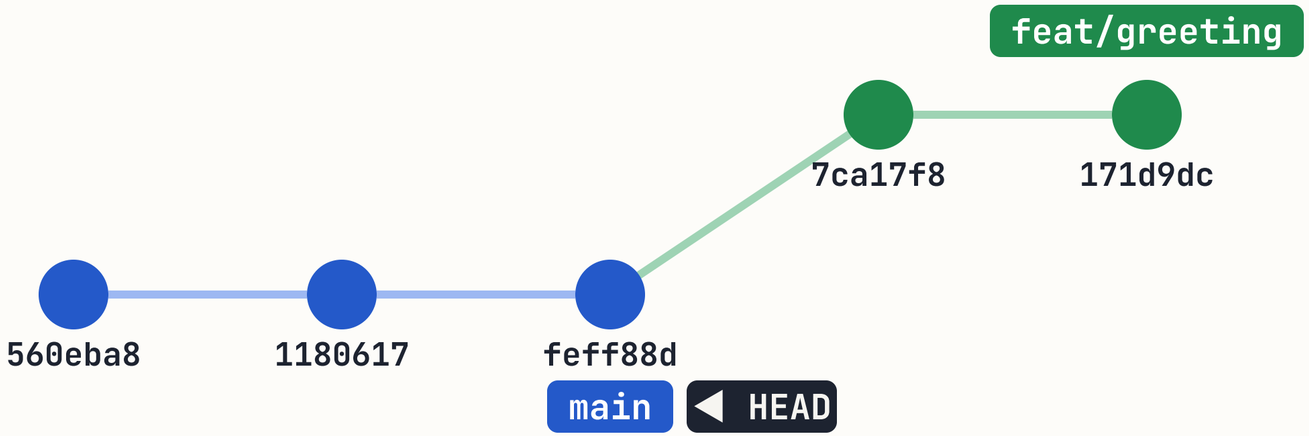


기능이 끝날 때까지 브랜치에서 여러 번 커밋해도 돼요.

만들고 옮기기

main으로 돌아가기

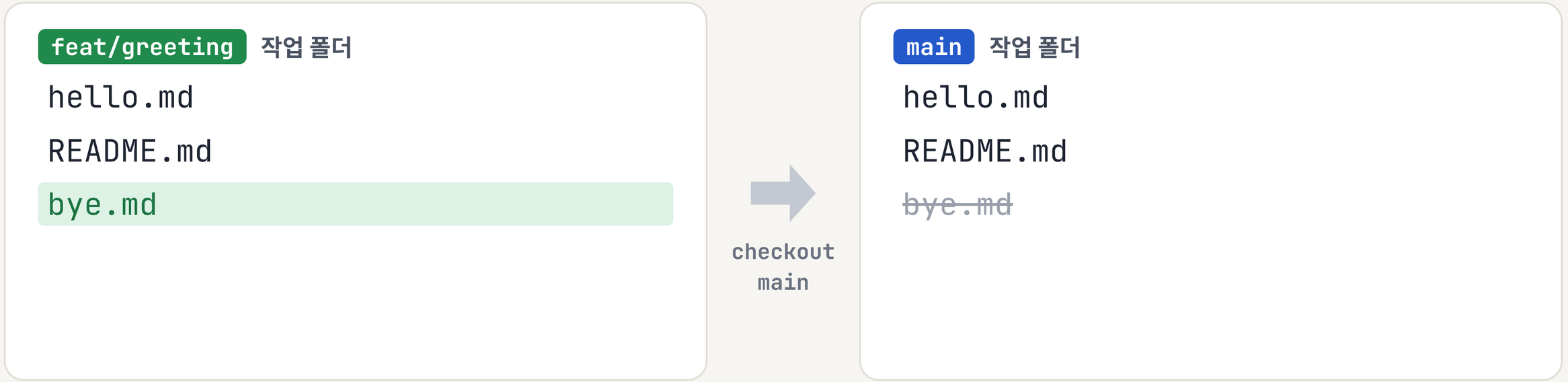
```
$ git checkout main
Switched to branch 'main'
```



HEAD만 main으로 돌아왔어요.

만들고 옮기기

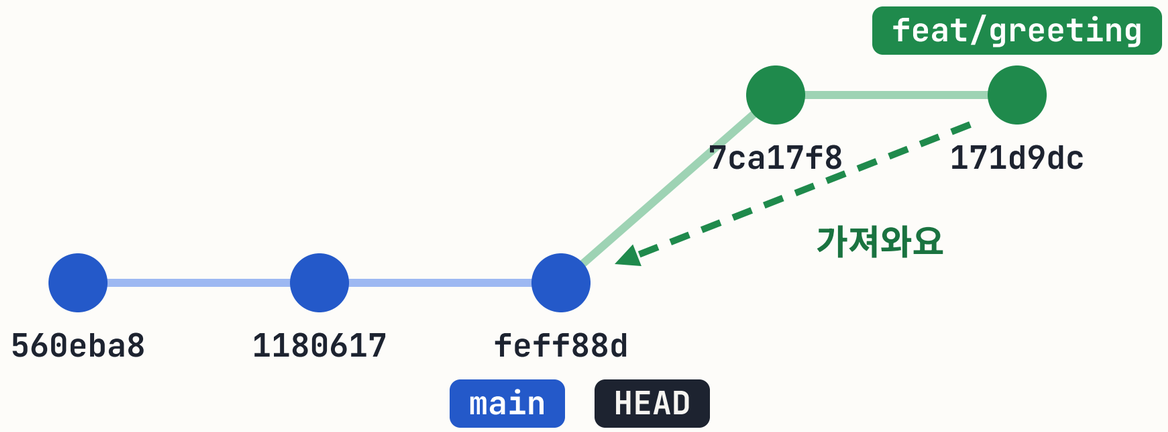
브랜치를 옮기면 폴더 파일도 바뀌어요



bye.md는 사라진 게 아니에요. feat/greeting으로 가면 다시 보여요.

합치기

merge는 받는 쪽에서 해요



받는 쪽

먼저 checkout으로 가 있는 브랜치

가져올 쪽

git merge 뒤에 적는 이름

1 **받을 브랜치로 이동**

```
git checkout main
```

2 **가져올 브랜치 이름을 적어요**

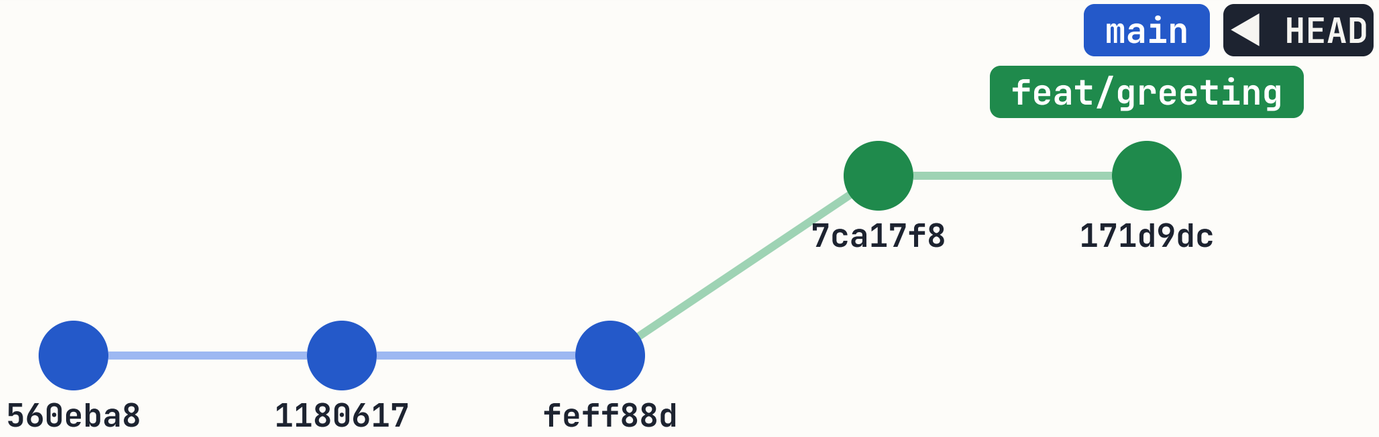
```
git merge feat/greeting
```

02에서 본 git pull origin main도 받아 온 커밋을 이 방식으로 merge해요.

합치기

Fast-forward: main 이름표만 앞으로

```
$ git merge feat/greeting
Updating feff88d..171d9dc
Fast-forward
```

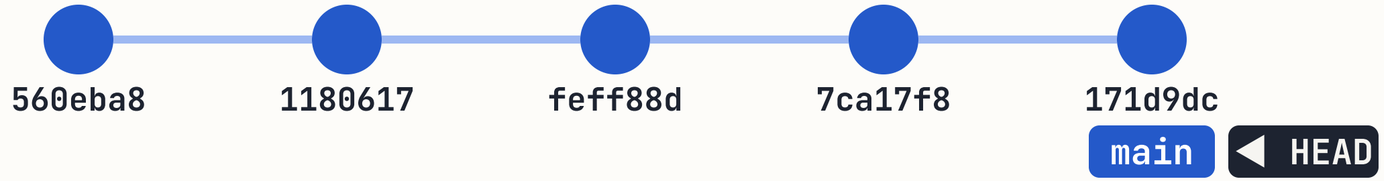


main 쪽에 새 커밋이 없으면 이름표만 당겨요.

합치기

다 쓴 브랜치 지우기

```
$ git branch -d feat/greeting
Deleted branch feat/greeting (was 171d9dc).
```



이름표만 떼요. 커밋은 main에 한 줄로 남아요.

합치기 · 브랜치 이름

브랜치 이름은 타입/하는 일

feat / login-button

타입 · 슬래시 · 하는 일(소문자, 하이픈)

feat/
새 기능
feat/login-button

fix/
버그 수정
fix/signup-error

docs/
문서
docs/readme-install

refactor/
구조 개선
refactor/user-api

chore/
설정, 패키지
chore/eslint-setup

타입은 커밋 메시지(feat, fix, docs …)와 같은 말을 써요.

합치기 · 브랜치 이름

이렇게 짓고, 이건 피해요

이렇게

`feat/login-button`

`fix/12-signup-error` (이슈 12번)

이건 피해요

`test`, `내작업`, `final2` (무슨 일인지 모름)

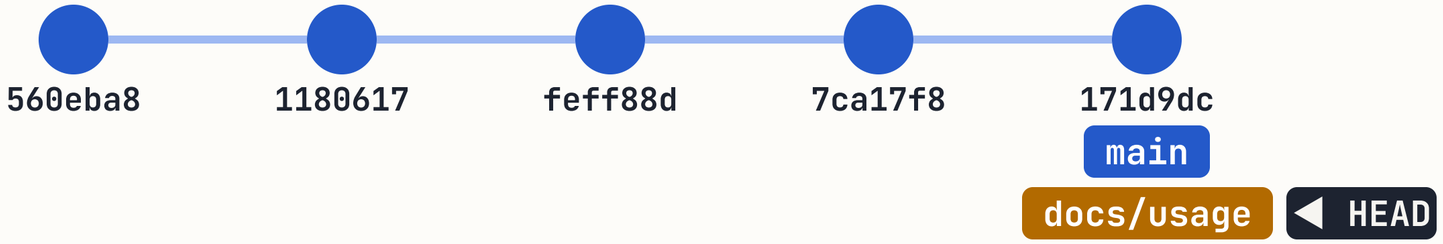
`Feat/Login Button` (대문자, 띄어쓰기)

소문자와 하이픈(-)만 쓰고, 무슨 일인지 이름에 적어요.

갈라진 길 합치기

새 브랜치 docs/usage 만들고 옮겨 가기

```
$ git checkout -b docs/usage
Switched to a new branch 'docs/usage'
```

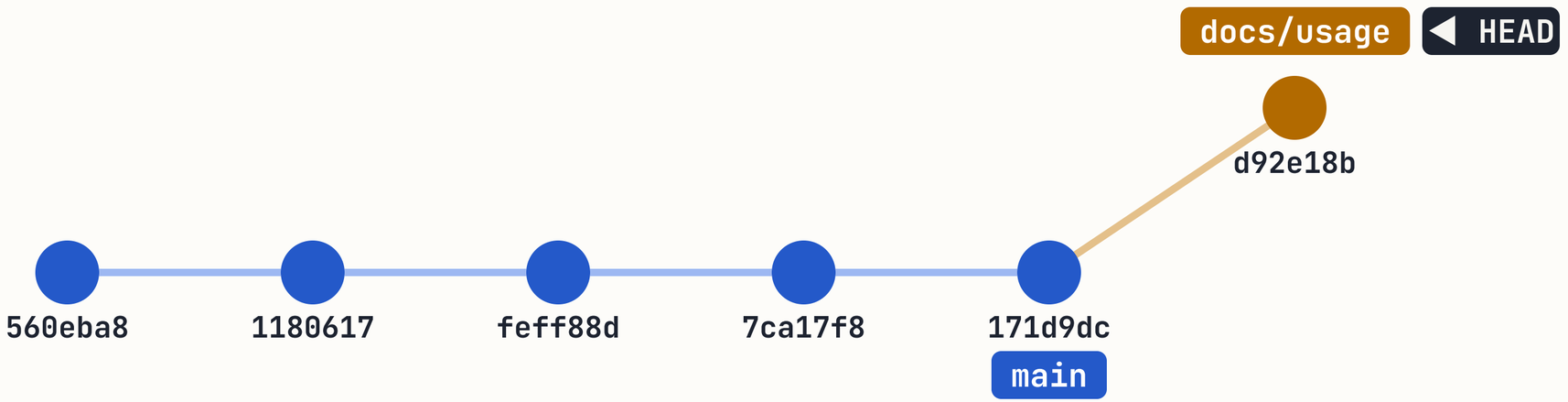


-b는 만들기와 옮기기를 한 번에 해요.

갈라진 길 합치기

docs/usage에서 커밋

```
$ git add README.md
$ git commit -m "docs: README에 소개 추가"
[docs/usage d92e18b] docs: README에 소개 추가
```

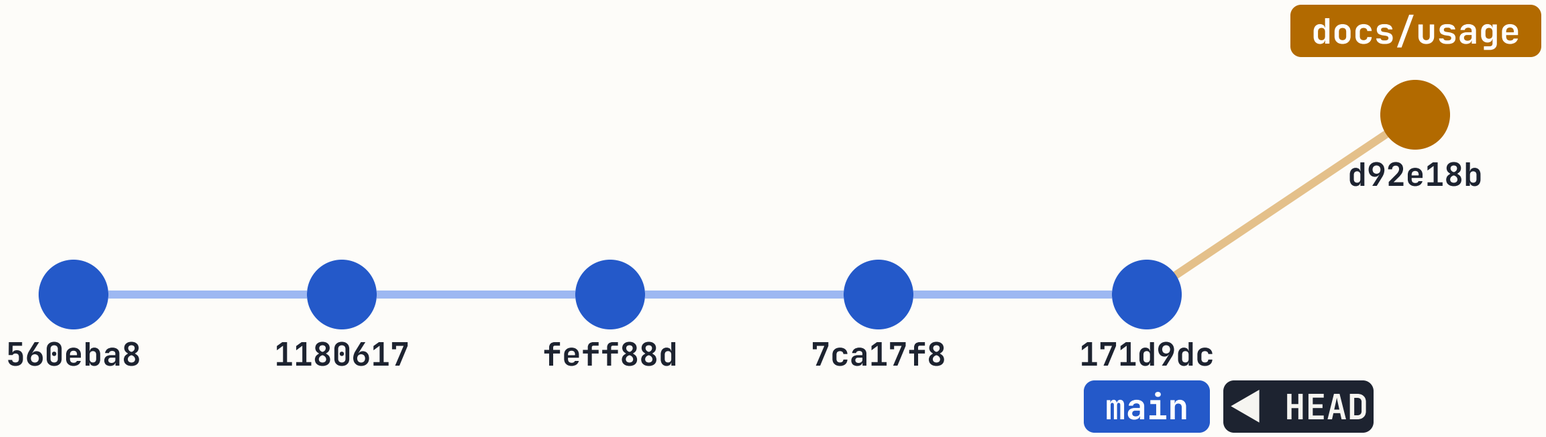


docs/usage만 한 칸 앞으로 가요.

갈라진 길 합치기

다시 main으로

```
$ git checkout main
Switched to branch 'main'
```

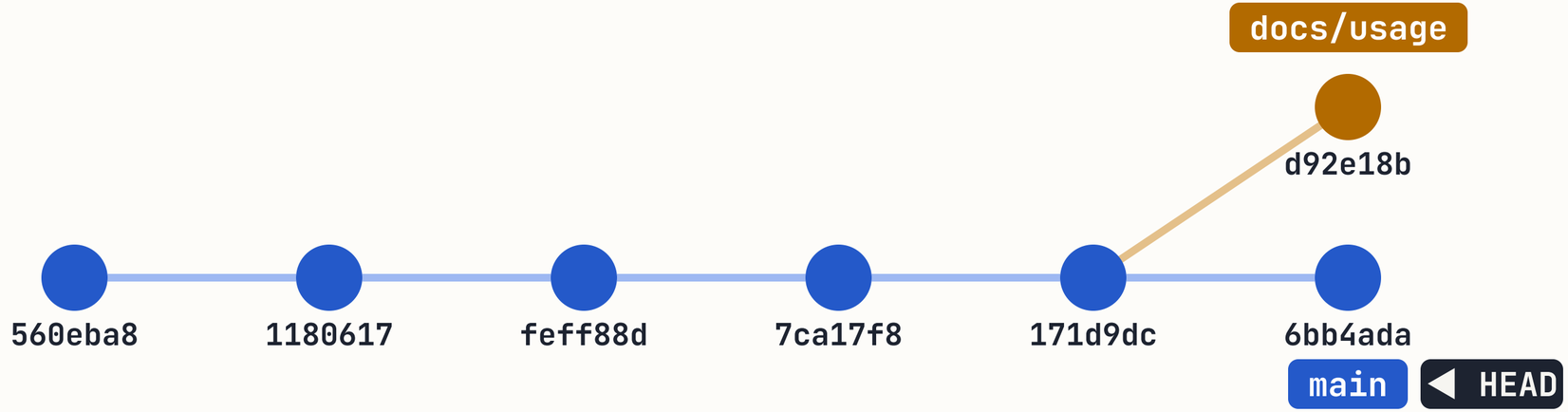


이번엔 main에서도 커밋해 볼게요.

갈라진 길 합치기

main에도 새 커밋: 길이 두 갈래가 됐어요

```
$ git add bye.md
$ git commit -m "feat: 끝인사 한 줄 추가"
[main 6bb4ada] feat: 끝인사 한 줄 추가
```

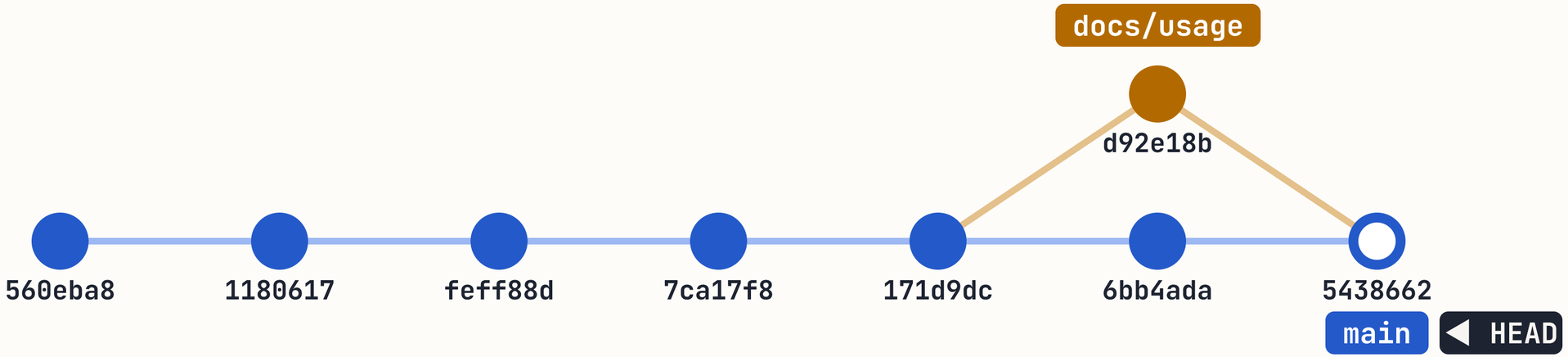


이번엔 이름표만 당겨서는 합칠 수 없어요.

갈라진 길 합치기

merge 커밋이 두 길을 이어요

```
$ git merge docs/usage
Merge made by the 'ort' strategy.
```

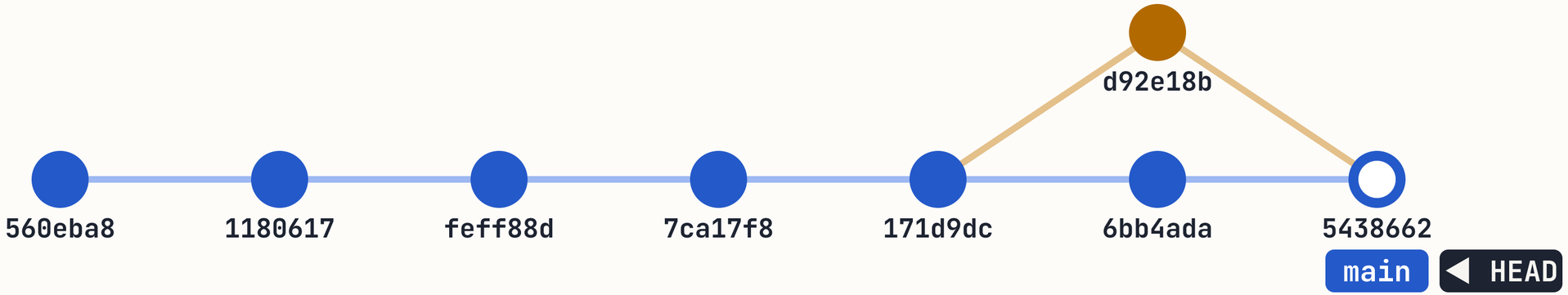


흰 점이 새로 생긴 merge 커밋이에요. 메시지 편집기가 열리면 그대로 저장하고 닫아요.

갈라진 길 합치기

docs/usage도 지우기

```
$ git branch -d docs/usage
Deleted branch docs/usage (was d92e18b).
```

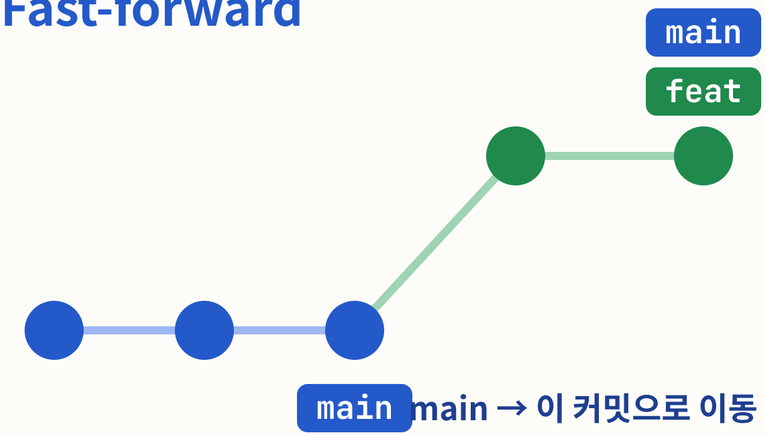


이름표만 사라지고 커밋은 main 기록 안에 남아요.

개념 정리

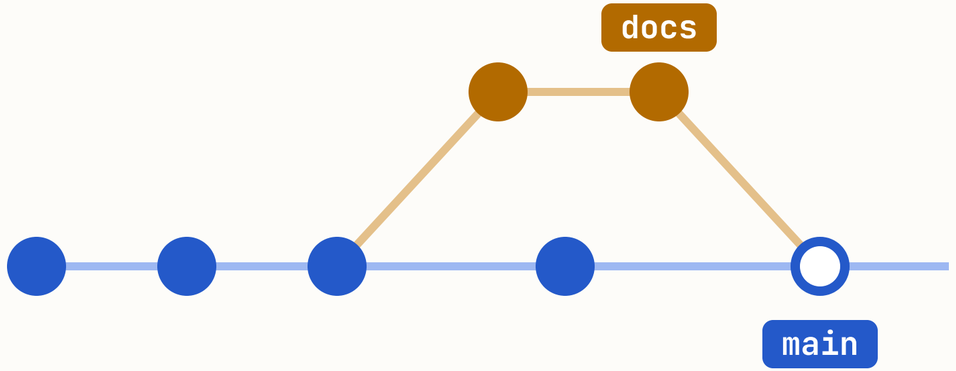
merge는 두 가지 모습으로 끝나요

Fast-forward



main에 새 커밋이 없을 때
이름표만 앞으로 당겨요

merge 커밋



양쪽에 새 커밋이 있을 때
둘을 잇는 커밋을 새로 만들어요

명령은 똑같이 git merge 브랜치이름. 어느 쪽이 될지는 Git이 정해요.

충돌 풀기

충돌은 언제 나요?

다른 파일을 고침
알아서 합쳐요

같은 파일, 멀리 떨어진 줄
알아서 합쳐요

같은 줄, 바로 옆 줄
충돌: 사람이 골라요

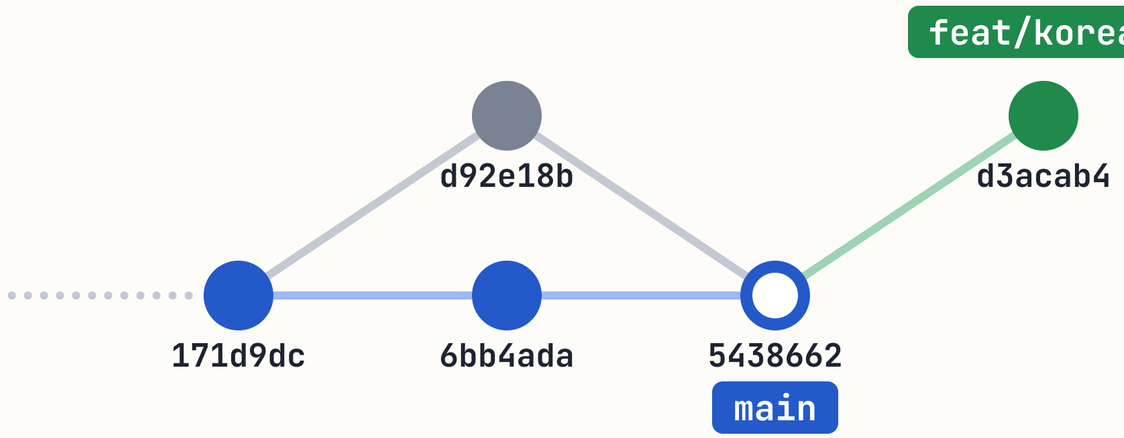
한쪽은 지우고, 한쪽은 고침
충돌: 사람이 골라요

충돌은 Git이 고를 수 없는 곳만 사람에게 물어보는 거예요.

충돌 시나리오 1 · 같은 줄

feat/korean에서 첫 줄을 안녕으로

```
$ git checkout -b feat/korean
$ git commit -am "feat: 인사를 한국어로"
[feat/korean d3acab4] feat: 인사를 한국어로
```



feat/korean의 hello.md

안녕

world

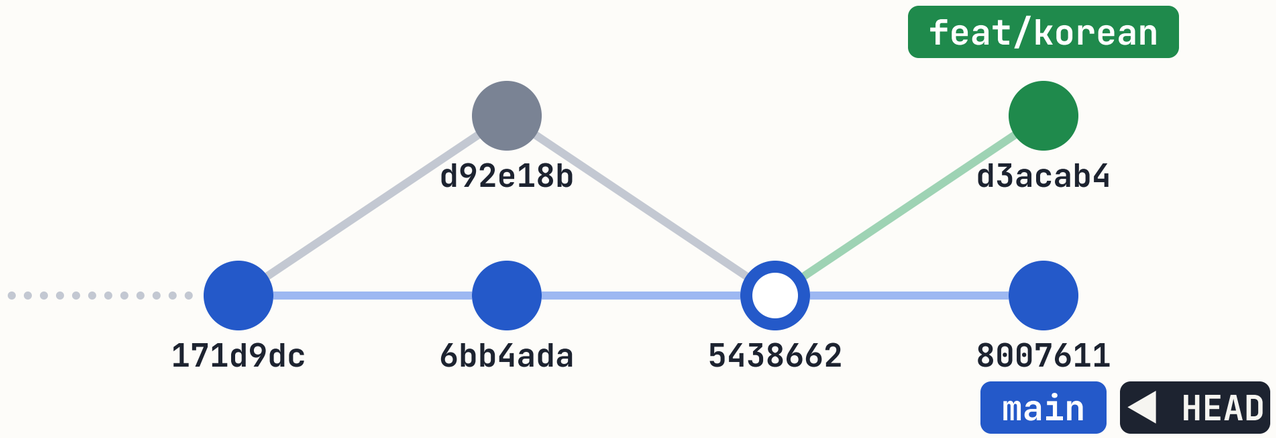
nice to meet you

hello.md 첫 줄 hello를 안녕으로 바꿨어요.

충돌 시나리오 1 · 같은 줄

main에서는 같은 줄을 hi로

```
$ git checkout main
$ git commit -am "feat: 인사를 짧게"
[main 8007611] feat: 인사를 짧게
```



main의 hello.md

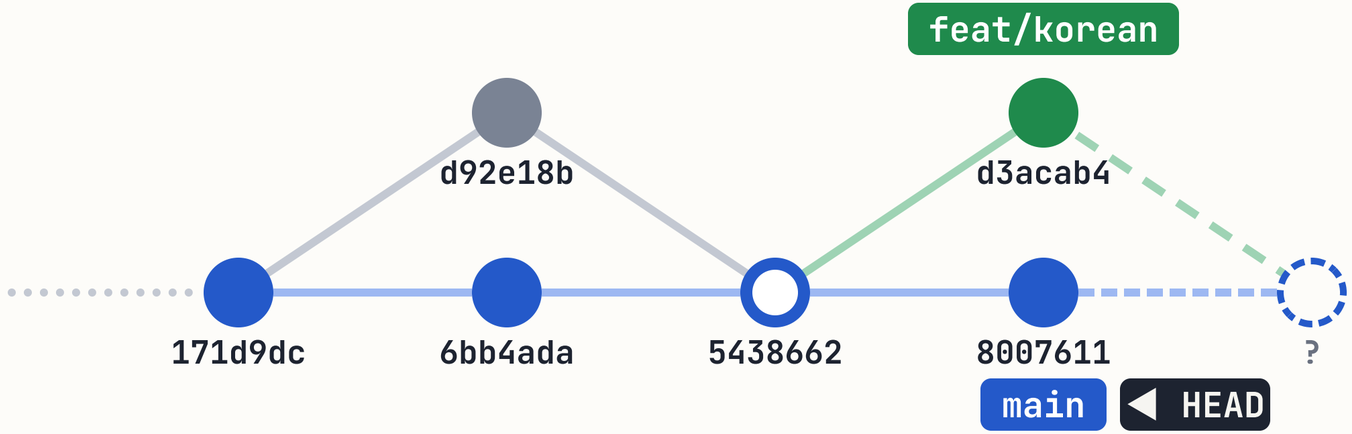
```
hi
world
nice to meet you
```

같은 줄을 양쪽에서 다르게 고쳤어요.

충돌 시나리오 1 · 같은 줄

합치려니 멈췄어요: 충돌

```
$ git merge feat/korean
CONFLICT (content): Merge conflict in hello.md
Automatic merge failed; fix conflicts and then commit the result.
```



Git이 hi와 안녕 중 무엇을 남길지 몰라서 나에게 물어보는 거예요.

충돌 풀기

충돌 표시 읽는 법

hello.md

```
<<<<<<< HEAD
hi
=======
안녕
>>>>>>> feat/korean
world
nice to meet you
```

지금 내 브랜치(main) 내용

합치려는 브랜치(feat/korean) 내용

푸는 순서

1 남길 내용으로 고치기
둘 다 남겨도 돼요

2 표시 세 줄 지우고 저장

3 `git add hello.md`

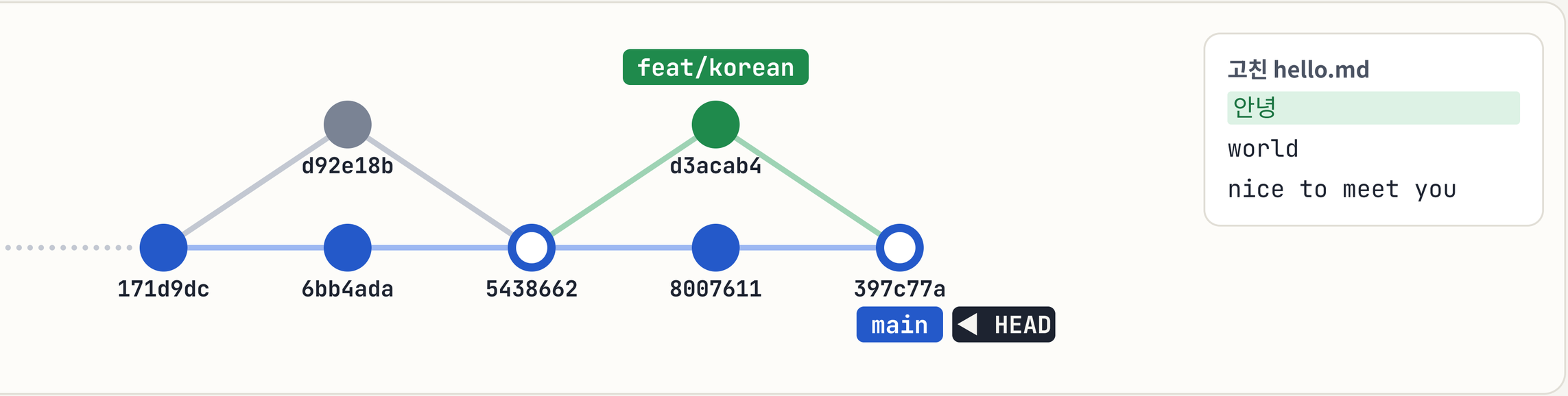
4 `git commit -m "..."`

<<<<<<<, =======, >>>>>>> 줄이 남은 채로 커밋하지 않아요.

충돌 시나리오 1 · 같은 줄

고치고 add, commit하면 끝

```
$ git add hello.md
$ git commit -m "fix: 인사 충돌 해결"
[main 397c77a] fix: 인사 충돌 해결
```



이 커밋이 두 길을 잇는 merge 커밋이에요.

충돌 풀기

헷갈리면 merge 전으로 돌아가요

```
$ git merge feat/korean
CONFLICT (content): Merge conflict in hello.md
Automatic merge failed; fix conflicts and then commit the result.
$ git merge --abort
$ git status
On branch main
nothing to commit, working tree clean
```

merge --abort는 합치기 전 모습으로 돌려놔요. 천천히 다시 해도 돼요.

GitHub에서 함께

팀에서는 이 순서로 일해요

내 컴퓨터

1

브랜치 만들기

```
git checkout -b feat/emoji
```

2

작업하고 커밋

```
add, commit
```

3

브랜치 올리기

```
git push origin feat/emoji
```

GitHub 웹사이트

4

합쳐 달라고 요청

```
Pull Request
```

5

확인하고 합치기

```
리뷰 → Merge
```

내 컴퓨터

6

내 main 맞추기

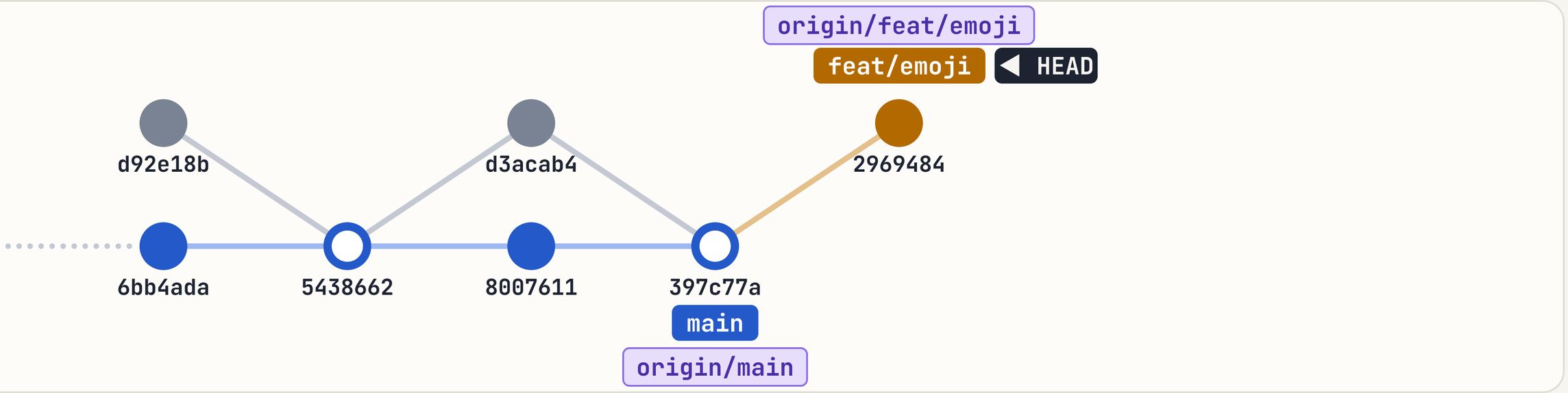
```
git checkout main
git pull origin main
```

main에는 직접 push하지 않고, 합치는 일은 Pull Request로 해요.

GitHub에서 함께

브랜치째 GitHub에 올려요

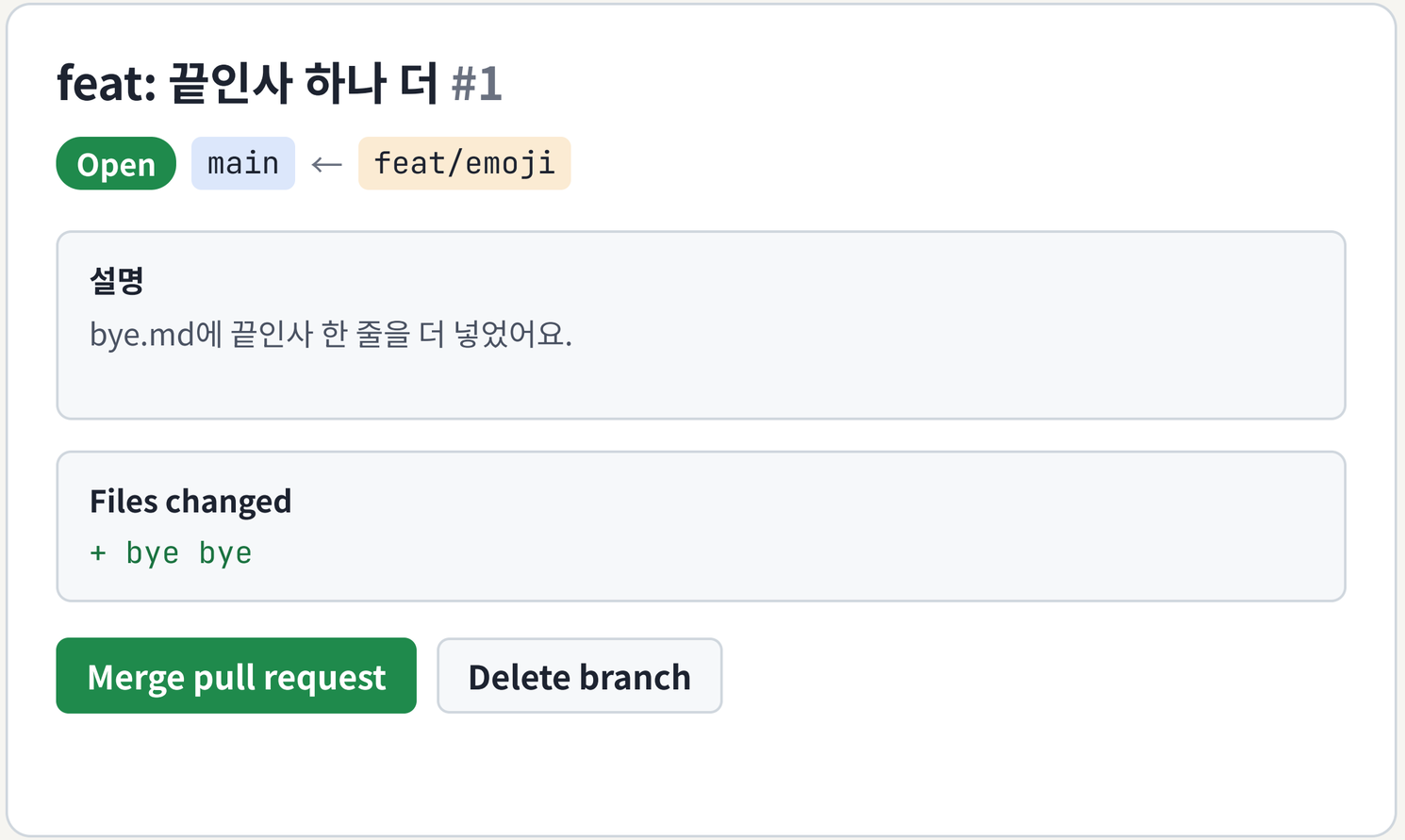
```
$ git checkout -b feat/emoji
$ git commit -am "feat: 끝인사 하나 더"
$ git push origin feat/emoji
 * [new branch] feat/emoji → feat/emoji
```



push할 때 main 대신 브랜치 이름을 끝까지 적어요.

GitHub에서 함께

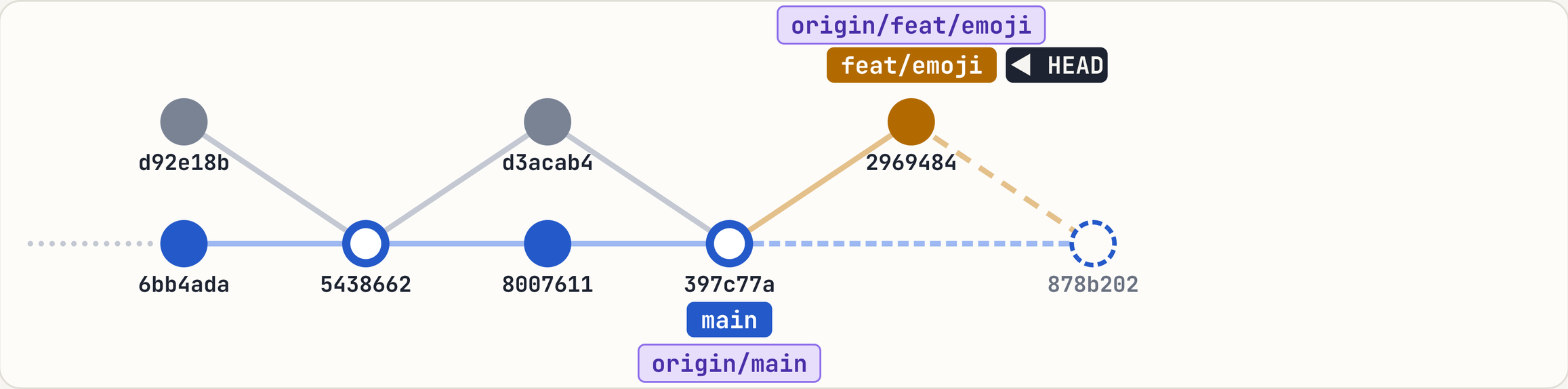
Pull Request 화면에서 할 일



1. 제목: 커밋 메시지처럼 타입: 요약
2. 어디로 ← 무엇을 합치는지
3. 무엇을 왜 바꿨는지
4. 바뀐 줄 확인, 댓글로 리뷰
5. 합치고 나면 GitHub 브랜치 삭제

GitHub에서 함께

GitHub에서 Merge하면 GitHub에만 커밋이 생겨요

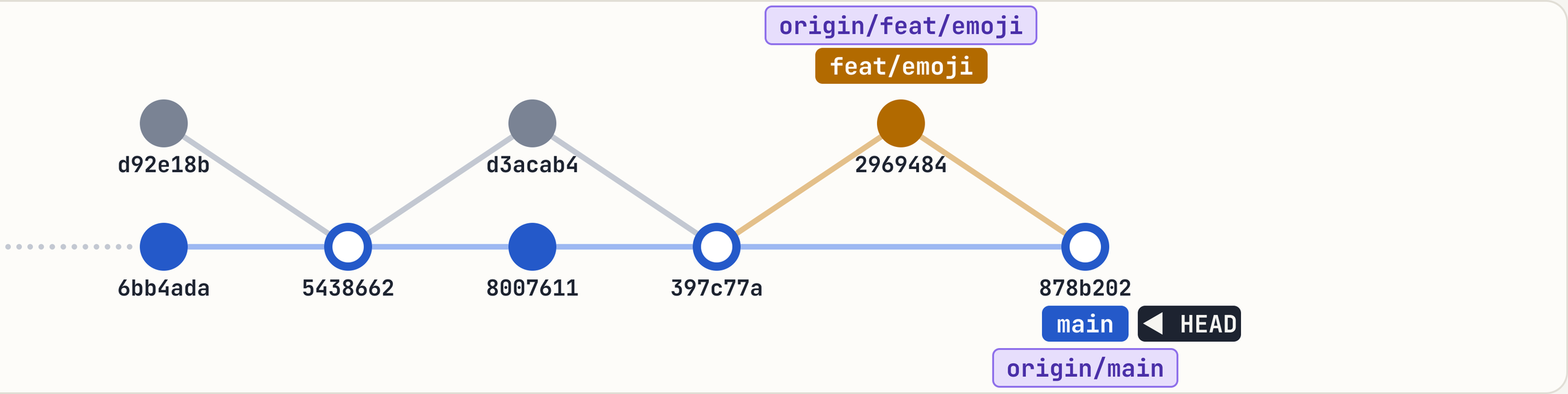


점선 커밋은 아직 GitHub에만 있어요. 내 컴퓨터는 몰라요.

GitHub에서 함께

main으로 가서 pull로 받아요

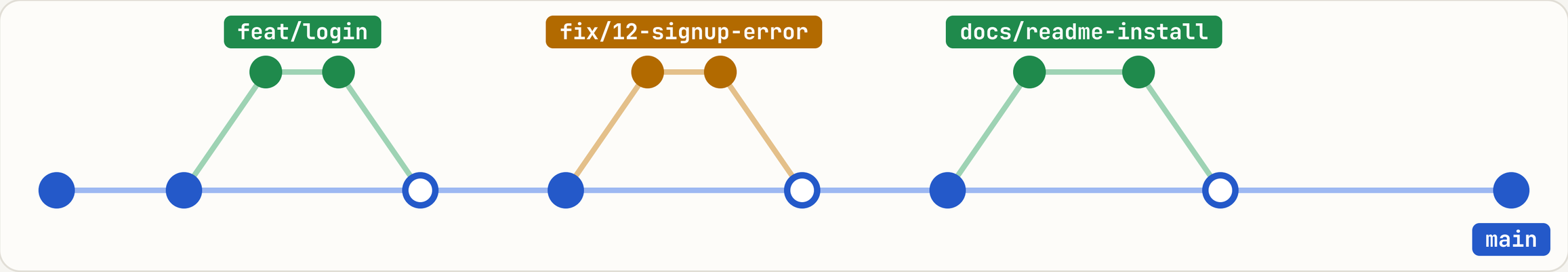
```
$ git checkout main
$ git pull origin main
Updating 397c77a..878b202
Fast-forward
```



다 쓴 feat/emoji는 git branch -d feat/emoji로 지워요.

GitHub에서 함께 · 브랜치 전략

우리는 GitHub Flow로 일해요



main은 늘 동작하는 상태.
직접 커밋하지 않아요

일 하나 = 브랜치 하나.
이름은 타입/하는 일

작게 자주 Pull Request

merge 뒤 브랜치는 지워요

Git Flow(develop, release 브랜치를 따로 두는 방식)는 배포 일정이 복잡한 회사에서 써요. 처음엔 GitHub Flow면 충분해요.

충돌 시나리오 2 · 바로 옆 줄

다른 줄인데도 충돌이 나요

main hello.md

```
hello
세상
```

feat/korean hello.md

```
안녕
world
```

```
$ git merge feat/korean
CONFLICT (content): Merge conflict in hello.md
Automatic merge failed; fix conflicts and then commit the result.
```

붙어 있는 줄을 각자 고치면 Git은 한 덩어리로 봐요.

충돌 시나리오 2 · 바로 옆 줄

둘 다 남기면 돼요

hello.md (충돌 표시)

```
<<<<<<< HEAD
hello
세상
=======
안녕
world
>>>>>>> feat/korean
```



main 고친 hello.md

안녕

세상

```
$ git add hello.md
$ git commit -m "fix: 인사 충돌 해결"
[main 1d3e1ba] fix: 인사 충돌 해결
```

양쪽에서 바뀐 줄을 하나씩 가져와 합치고, 표시 세 줄은 지워요.

충돌 시나리오 3 · 지움 vs 고침

한쪽은 파일을 지우고, 한쪽은 고쳤어요

main bye.md

```
see you
bye
```

chore/remove-bye bye.md

~~파일 삭제~~

```
$ git merge chore/remove-bye
CONFLICT (modify/delete): bye.md deleted in chore/remove-bye and modified in HEAD.
Automatic merge failed; fix conflicts and then commit the result.
```

파일 안에 충돌 표시가 없어요. 지울지 남길지만 정하면 돼요.

충돌 시나리오 3 · 지움 vs 고침

지울지 남길지 골라요

```
$ git status
Unmerged paths:
(use "git add/rm <file>..." as appropriate to mark resolution)
    deleted by them: bye.md
```

⚠ 지우기로 하면

```
git rm bye.md
git commit -m "..."
```

✓ 남기기로 하면

```
git add bye.md
git commit -m "..."
```

deleted by them은 합치려는 브랜치(them)가 지웠다는 뜻이에요.

충돌 시나리오 4 · 같은 이름 새 파일

둘 다 todo.md를 새로 만들었어요

main todo.md (새 파일)

```
- 운동
```

feat/todo todo.md (새 파일)

```
- 장보기
```

```
$ git merge feat/todo
CONFLICT (add/add): Merge conflict in todo.md
Automatic merge failed; fix conflicts and then commit the result.
```

처음부터 내용이 달라서, 파일 전체가 충돌 표시로 묶여요.

충돌 시나리오 4 · 같은 이름 새 파일

목록이면 둘 다 살려요

todo.md (충돌 표시)

```
<<<<<<< HEAD
- 운동
=======
- 장보기
>>>>>>> feat/todo
```



main 고친 todo.md

- 운동
- 장보기

```
$ git add todo.md
$ git commit -m "fix: 할 일 합치기"
[main fcf74a9] fix: 할 일 합치기
```

푸는 방법은 시나리오 1과 같아요. 고르고, 표시 지우고, add, commit.

충돌 시나리오 5 · pull 할 때

팀원이 먼저 같은 줄을 push했어요

origin/main GitHub의 hello.md (팀원)

```
hi
world
```

main 내 hello.md

```
안녕
world
```

```
$ git push origin main
! [rejected] main → main (fetch first)
$ git pull origin main
fatal: Need to specify how to reconcile divergent branches.
```

갈라진 상태에서 pull하면 어떻게 합칠지 정해 달라며 멈춰요.

충돌 시나리오 5 · pull 할 때

처음 한 번만 설정해요

```
git config --global pull.rebase false
```

git pull

pull은 두 단계예요

= fetch

GitHub 커밋을 받아 오고

+ merge

내 main과 합쳐요

pull.rebase false는 pull할 때 merge로 합치라는 뜻이에요. 설정한 뒤 다시 pull해요.

충돌 시나리오 5 · pull 할 때

다시 pull하면 익숙한 충돌이에요

```
$ git pull origin main
CONFLICT (content): Merge conflict in hello.md
```

hello.md (충돌 표시)

```
<<<<<<< HEAD
안녕
=======
hi
>>>>>>> 27dcbf520a13a80a5291bda2068065ad3623b989
```

```
$ git add hello.md
$ git commit -m
"fix: 팀원과 인사 충돌 해결"
$ git push origin main
27dcbf5..d524032 main → main
```

pull 충돌은 아래쪽 이름이 브랜치 대신 커밋 해시예요. 푸는 법은 같아요.

충돌 시나리오 6 · Pull Request

PR 화면에 충돌이 떠요

feat: 인사를 한국어로 #2

Open `main` ← `feat/korean`

⚠ **This branch has conflicts that must be resolved**

`Conflicting files: hello.md`

Merge pull request

내가 PR을 올린 사이 main이 같은 줄을 바꿨어요. Merge 버튼이 막혀요.

충돌 시나리오 6 · Pull Request

내 브랜치에서 main을 받아 풀고, 다시 push

```
$ git checkout feat/korean
$ git pull origin main
CONFLICT (content): Merge conflict in hello.md
# hello.md 충돌 표시 지우고 저장
$ git add hello.md
$ git commit -m "fix: main과 인사 충돌 해결"
$ git push origin feat/korean
900b045..2aeefa6 feat/korean → feat/korean
```

push하면 PR 화면이 다시 Merge 할 수 있는 상태로 바뀌어요.

충돌 시나리오

메시지로 충돌 종류를 알아봐요

`CONFLICT (content)`	**같은 줄, 옆 줄**	고르고, 표시 지우고, add, commit
`CONFLICT (modify/delete)`	**지움 vs 고침**	git rm 또는 git add, commit
`CONFLICT (add/add)`	**같은 이름 새 파일**	고르고, 표시 지우고, add, commit
`fatal: Need to specify how to reconcile ...`	**갈라진 채 pull**	git config --global pull.rebase false
`This branch has conflicts ...`	**PR 충돌**	내 브랜치에서 pull origin main, 풀고 push

어떤 충돌이든 막히면 git merge --abort로 처음으로 돌아갈 수 있어요.

에러 1 · 브랜치 이름

옮기려는데, 만들려는데 안 돼요

없는 브랜치로 옮김

```
$ git checkout feat/nope
error: pathspec 'feat/nope' did not match any
file(s) known to git
```

이름 오타이거나 아직 안 만든 브랜치예요

```
git branch
```

```
git checkout -b feat/nope
```

이미 있는 이름으로 만듦

```
$ git checkout -b feat/a
fatal: a branch named 'feat/a' already exists
```

같은 이름이 벌써 있어요. -b 없이 옮겨 가요

```
git checkout feat/a
```

에러 2 · 커밋 안 한 변경

브랜치를 옮기려는데 막혀요

고친 파일이 그대로 있음

```
error: Your local changes to the following files would be overwritten by checkout:
        hello.md
Please commit your changes or stash them before you switch branches.
Aborting
```

옮겨 가면 고친 내용이 덮어써져서 Git이 멈췄어요. 먼저 커밋하고 옮겨요

```
git add hello.md
```

```
git commit -m "feat: ..."
```

```
git checkout feat/b
```

에러 3 · 브랜치 지우기

브랜치가 안 지워져요

아직 합치지 않은 커밋이 있음

```
$ git branch -d feat/a
error: the branch 'feat/a' is not fully merged.
If you are sure you want to delete it, run 'git branch -D feat/a'
```

지우면 그 커밋을 잃을 수 있어서 Git이 멈췄어요

```
git checkout main
```

```
git merge feat/a
```

```
git branch -d feat/a
```

한 장 요약

브랜치

```
git branch
git branch 이름
git checkout 이름
git checkout -b 이름
git branch -d 이름
```

합치기

```
git checkout main
git merge 이름
```

충돌

```
파일에서 고르기
표시 세 줄 지우기
git add 파일
git commit -m "..."
막히면 git merge --abort
```

GitHub과 함께

```
git push origin 이름
→ Pull Request
→ Merge
git checkout main
git pull origin main
```

브랜치 이름: feat/login-button · main에는 직접 커밋하지 않기